Metaheurísticas

Aula 2 — No Free Lunch, Classificação e Métricas de Desempenho

Raoni F. S. Teixeira

Objetivos desta aula

Ao final desta aula, o aluno deve ser capaz de:

1. Enunciar e interpretar o Teorema *No Free Lunch* e extrair suas consequências práticas para a escolha de algoritmos;
2. Classificar metaheurísticas segundo taxonomia padronizada;
3. Compreender o *trade-off* exploração/explotação como eixo estruturante dos algoritmos;
4. Aplicar métricas estatísticas rigorosas para comparar algoritmos de otimização;
5. Conduzir experimentos computacionais reprodutíveis com protocolo estatístico correto.

1 O Teorema No Free Lunch

Na Aula 1, concluímos que heurísticas específicas — como a regra gulosa do Problema da Mochila — falham porque ficam presas em bacias de atração locais. A solução proposta foi usar metaheurísticas: estratégias agnósticas ao problema, capazes de escapar dessas armadilhas. Mas isso levanta uma nova pergunta: **entre as dezenas de metaheurísticas existentes, existe alguma universalmente superior às outras?**

A resposta é formalmente negativa, e essa negação tem nome próprio.

1.1 Formulação

Antes de enunciar o teorema, é necessário precisar o escopo de sua aplicação. O resultado se refere exclusivamente a **algoritmos de caixa-preta** (*black-box*):

Definição — Algoritmo de Otimização de Caixa-Preta

Um algoritmo de otimização é dito de **caixa-preta** se a única informação que utiliza sobre a função objetivo f provém de avaliações pontuais $f(\boldsymbol{x})$ — sem acesso a derivadas, estrutura analítica, convexidade ou qualquer outra propriedade da função.

Metaheurísticas são, por definição, algoritmos de caixa-preta. Métodos que exploram estrutura do problema — como Newton-Raphson (usa gradiente e Hessiana) ou o Simplex (explora convexidade) — estão **fora do escopo do NFL**. O teorema não invalida esses métodos em seus domínios de aplicabilidade.

Teorema — No Free Lunch — Wolpert & Macready, 1997

Sejam $f : \mathcal{X} \to \mathcal{Y}$ uma função objetivo e a_1, a_2 dois algoritmos de otimização de caixa-preta quaisquer. Se tomarmos a média do desempenho de a_1 e a_2 sobre **todas** as funções $f : \mathcal{X} \to \mathcal{Y}$ (considerando a distribuição uniforme sobre o espaço de funções), então:

$$\sum_f P(d_m^y \mid f, m, a_1) = \sum_f P(d_m^y \mid f, m, a_2)$$

onde d_m^y é a sequência de valores de f observados nas m primeiras iterações. Em palavras: **quaisquer dois algoritmos de caixa-preta têm performance média idêntica quando a média é tomada sobre todos os problemas possíveis.**

O resultado foi provado por Wolpert e Macready (1997) e generalizado em trabalhos posteriores. Ele se aplica tanto a otimização quanto a aprendizado supervisionado.

⚠️ **Atenção**

Precisão sobre "busca aleatória". É comum, mas impreciso, enunciar o NFL como "nenhum algoritmo é melhor que busca aleatória em média". A afirmação correta é mais forte: **todos** os algoritmos de caixa-preta têm exatamente a mesma performance média agregada sobre o espaço de todas as funções — busca aleatória inclusa. A busca aleatória não é o referencial da prova; ela é apenas um caso particular que torna a conclusão mais intuitiva.

1.2 Interpretação intuitiva

A prova formal é sofisticada, mas a intuição é acessível. Imagine o espaço de todas as funções $f : \{1, \ldots, N\} \to \mathbb{R}$. Para cada função onde o algoritmo a_1 se sai bem, existe uma função simétrica onde ele se sai mal. A "vantagem" conquistada em certas funções é exatamente compensada pela "desvantagem" em outras. Nenhum algoritmo tem vantagem **incondicional**.

⚠️ **Atenção**

O NFL não diz que "todos os algoritmos são iguais em problemas práticos". Diz que, sem conhecimento sobre a estrutura do problema, não há razão *a priori* para preferir um algoritmo de caixa-preta a outro. A vantagem de qualquer método provém de **hipóteses implícitas sobre a estrutura do problema** — e essas hipóteses devem ser explicitadas e verificadas empiricamente.

1.3 Consequências práticas

O NFL tem implicações diretas e profundas para qualquer aplicação de metaheurísticas:

1. Não existe algoritmo universalmente superior. Um PSO calibrado para um problema contínuo pode ser inferior a um AG simples num problema combinatório com a mesma dimensão. Transferir resultados entre classes de problemas sem justificativa estrutural é metodologicamente inadequado — independentemente do domínio de aplicação.

2. Conhecimento do domínio é a vantagem real. Representações, operadores e parâmetros específicos ao problema são o que diferenciam implementações eficazes de implementações genéricas. Quem conhece a estrutura do problema pode projetar operadores que preservem viabilidade ou explorem regularidades da paisagem — e isso produz ganhos mensuráveis de desempenho que a teoria não proíbe, pois não são algoritmos puramente de caixa-preta.

3. Comparações precisam ser restritas a classes de problemas. Afirmar que "o algoritmo X é melhor" só é válido se a comparação for conduzida sobre um conjunto representativo de instâncias daquela **classe específica**, com metodologia estatística adequada e protocolo reprodutível.

4. Hibridização é racionalmente justificada. Se nenhum algoritmo de caixa-preta é superior em geral, combinar métodos que fazem hipóteses complementares sobre a estrutura do problema é uma estratégia coerente com o NFL — e frequentemente eficaz na prática.

Corolário — NFL e escolha de algoritmos
A escolha de uma metaheurística deve ser **guiada pela estrutura do problema**. Os critérios relevantes são universais — aplicam-se ao Problema da Mochila, ao roteamento de veículos, ao projeto de circuitos ou ao despacho de energia:

- **Natureza das variáveis**: contínua, inteira, mista (MINLP).
- **Topologia da paisagem de busca**: unimodal, multimodal, rugosa, com platôs.
- **Viabilidade**: se as restrições são fortes, o algoritmo precisa lidar com viabilidade por penalidade, reparo ou representação especial — e isso afeta diretamente qual família de métodos é aplicável.
- **Custo de avaliação de f**: avaliações caras restringem o orçamento e desfavorecem métodos que avaliam f muitas vezes por iteração (ver Seção 3.4).
- **Requisitos de latência**: aplicações em tempo real exigem convergência em segundos, não minutos — o que pode ser mais determinante que a qualidade da solução.

Não existe "melhor algoritmo" sem essa especificação de contexto.

2 Taxonomia de Metaheurísticas

Metaheurísticas são estratégias de alto nível para guiar heurísticas de busca, independentes do problema específico (Glover, 1986; Blum & Roli, 2003). A literatura acumulou dezenas de propostas nos últimos 40 anos, e uma taxonomia clara ajuda a navegar esse espaço. Na Aula 1 introduzimos as duas grandes famílias (trajetória vs. população) e os algoritmos do curso. Esta seção aprofunda os critérios de classificação e, em especial, o trade-off exploração/explotação — que é o eixo estruturante de **todos** os algoritmos que estudaremos.

2.1 Critérios de classificação

Quatro eixos organizam o espaço de metaheurísticas:

Solução única vs. baseados em população. Algoritmos de **trajetória** (Hill Climbing, Simulated Annealing, Busca Tabu) mantêm e refinam uma única solução candidata. Algoritmos **populacionais** (Algoritmos Genéticos, PSO, ACO, DE) mantêm um conjunto de soluções que evoluem coletivamente, permitindo exploração paralela de múltiplas regiões do espaço de busca.

Memória. Algoritmos sem memória (Hill Climbing, SA básico) tomam decisões com base apenas no estado corrente. Algoritmos com memória de curto prazo (Busca Tabu) registram movimentos recentes para evitar ciclagem. Algoritmos com memória de longo prazo registram padrões de busca para guiar exploração futura — a distinção mais importante para entender por que a Busca Tabu supera o Hill Climbing em paisagens rugosas.

Inspiração. Muitas metaheurísticas são inspiradas em fenômenos naturais: evolução biológica (AG, DE), comportamento coletivo (PSO, ACO), processos físicos (SA). Essa inspiração motiva a escolha de operadores, mas **não garante desempenho superior** — afirmação que exige validação empírica rigorosa e é frequentemente negligenciada na literatura. O NFL torna isso um imperativo metodológico, não apenas uma recomendação.

Determinismo. Algoritmos determinísticos (Hill Climbing clássico) produzem sempre o mesmo resultado para o mesmo ponto de partida. Algoritmos estocásticos (SA, AG, PSO) incorporam aleato-

riedade — essencial para escapar de mínimos locais, mas que impõe a necessidade de análise estatística rigorosa dos resultados (Seção 3).

Algoritmo	Estrutura	Memória	Objetivos	Aula
Hill Climbing	Trajetória	Nenhuma	Mono	3
Simulated Annealing	Trajetória	Temperatura (curto prazo)	Mono	3
Busca Tabu	Trajetória	Lista tabu (curto+longo)	Mono	4
Algoritmo Genético	População	Genoma (implícita)	Mono/Multi	5
Differential Evol.	População	Vetores da pop.	Mono/Multi	5
PSO	População	Melhor pessoal e global	Mono/Multi	6
ACO	População	Feromônio (longo prazo)	Mono/Multi	6
NSGA-II	População	Frente de Pareto	Multi	7

Table 1: Taxonomia dos algoritmos estudados neste curso. A coluna "Estrutura" descreve a organização interna do método; "Objetivos" indica se o algoritmo foi originalmente projetado para otimização mono-objetivo, multi-objetivo, ou admite extensões para ambos.

2.2 O Trade-off Exploração-Explotação

O eixo central de qualquer metaheurística é o equilíbrio entre **exploração** (*exploration*) e **explotação** (*exploitation*). Compreender esse dilema é mais importante do que memorizar algoritmos específicos.

2.2.1 A Intuição: Dois Extremos Que Falham

- **Exploração**: investigar novas regiões do espaço de busca, mesmo que isso leve a soluções temporariamente piores. Reduz o risco de convergência prematura para mínimos locais.
- **Explotação**: concentrar-se na vizinhança de boas soluções já encontradas para refiná-las. Permite convergir rapidamente, mas aumenta o risco de perder o ótimo global.

O Problema da Mochila da Aula 1 ilustra os dois extremos com precisão. A **heurística gulosa** é explotação pura: ela segue a melhor direção local (maior $\frac{v_i}{w_i}$) sem jamais reconsiderar. O resultado é uma solução de R$ 21 — boa, mas não ótima. A **busca aleatória pura** é exploração pura: ela amostra o espaço sem direcionar a busca, e raramente encontra o ótimo R$ 22 em instâncias maiores.

A dificuldade fundamental é que nenhum algoritmo tem acesso ao mapa global da paisagem de busca: não sabe se a solução atual é um mínimo local profundo ou apenas um vale raso com regiões melhores a poucos passos. Toda metaheurística é uma resposta diferente à pergunta: **como usar a informação de soluções já avaliadas para decidir onde explorar a seguir?**

2.2.2 Mecanismos nos Algoritmos

Todo algoritmo de busca implementa esse balanço por meio de parâmetros e operadores específicos. Antes de apresentar o mapeamento, vale discutir o papel do cruzamento, que é frequentemente mal interpretado.

O **cruzamento** (recombinação) em Algoritmos Genéticos e DE não tem papel fixo no espectro exploração/explotação: ele depende do operador específico e de seus parâmetros. Operadores como o BLX-α com α grande têm caráter predominantemente exploratório, pois podem gerar descendentes fora do intervalo delimitado pelos pais. Operadores SBX com distribuição estreita (η grande) têm caráter

de explotação. Essa dependência paramétrica é uma fonte comum de confusão na literatura e deve ser levada em conta ao projetar e reportar experimentos.

Mecanismo	Papel predominante	Algoritmo
Temperatura alta	Exploração	SA
Temperatura baixa	Explotação	SA
Taxa de mutação alta	Exploração	AG, DE
Cruzamento BLX-α largo	Exploração	AG, DE
Cruzamento SBX estreito	Explotação	AG, DE
Lista tabu	Exploração (evita revisitas)	Busca Tabu
Evaporação de feromônio	Exploração	ACO
Feromônio concentrado	Explotação	ACO
Peso de inércia ω alto	Exploração	PSO
Coef. cognitivo/social altos	Explotação	PSO

Table 2: Mapeamento de mecanismos algorítmicos para os polos de exploração e explotação. O papel do cruzamento depende do operador e de seus parâmetros — ver texto.

2.3 Intensificação e Diversificação

Na Busca Tabu e em metaheurísticas de trajetória, os termos usados são **intensificação** (equivalente a explotação) e **diversificação** (equivalente a exploração). A Busca Tabu implementa estratégias explícitas para ambas:

- **Intensificação**: ao encontrar uma região promissora, o algoritmo concentra a busca nela, explorando sua vizinhança com maior detalhe.
- **Diversificação**: quando a busca estagna, o algoritmo força movimentos para regiões não visitadas, usando memória de longo prazo para identificá-las.

Essa dualidade é universal: qualquer metaheurística eficaz precisa alternar, ao longo da execução, entre fases de exploração e de explotação. A pergunta central de projeto é **como** e **quando** fazer essa transição — e a resposta depende, inevitavelmente, da estrutura do problema.

3 Métricas de Desempenho e Análise Experimental

Dado o NFL, a única forma de comparar algoritmos é **empiricamente**, por meio de experimentos rigorosos. Esta seção apresenta as métricas e o protocolo experimental que usaremos ao longo do curso.

3.1 Métricas básicas

Sejam N_r execuções independentes de um algoritmo sobre uma instância de problema. Cada execução r produz:

- f_r^*: melhor valor de f encontrado na execução r;
- t_r: tempo computacional da execução r (em segundos ou número de avaliações de f);
- k_r: iteração em que f_r^* foi encontrado.

As métricas agregadas são:

$$\overline{f^*} = \frac{1}{N_r}\sum_r f_r^*, \quad \sigma_{f^*} = \sqrt{\frac{1}{N_r - 1}\sum_r \left(f_r^* - \overline{f^*}\right)^2}$$

$$\text{Taxa de sucesso} = \frac{|\{r : f_r^* \leq f^{\text{ref}} + \varepsilon\}|}{N_r}$$

onde f^{ref} é um valor de referência conhecido (ótimo global, se disponível, ou melhor resultado da literatura) e ε é uma tolerância absoluta ou relativa.

> ⚠️ **Atenção**
> **Nunca compare algoritmos com base em uma única execução.** Algoritmos estocásticos têm variância intrínseca; um único resultado não é representativo do comportamento do método. A prática consolidada na literatura exige pelo menos $N_r = 30$ execuções com sementes aleatórias distintas e **registradas** (para reprodutibilidade). Com amostras menores, o poder estatístico dos testes não-paramétricos usuais (Wilcoxon, Friedman) é insuficiente para detectar diferenças relevantes, exceto quando o tamanho do efeito é muito grande.

3.2 Curva de convergência

A **curva de convergência** mostra a evolução de f^* (melhor solução encontrada até o momento) em função do número de avaliações da função objetivo. É a ferramenta diagnóstica mais informativa para entender o comportamento de um algoritmo.

Uma curva de convergência típica tem três fases:
1. **Exploração inicial**: queda rápida de f^* nas primeiras iterações.
2. **Refinamento**: queda mais lenta, à medida que o algoritmo se concentra em regiões promissoras.
3. **Estagnação**: platô onde o algoritmo não consegue mais melhorar (convergência para ótimo local ou esgotamento do orçamento).

Ao comparar dois algoritmos, um pode convergir mais rápido (melhor fase 1) mas para um valor pior (fase 3 inferior). Qual é preferível depende do orçamento computacional disponível e dos requisitos da aplicação.

> ⚠️ **Atenção**
> **Cálculo do IC 95% na curva de convergência.** Para computar um intervalo de confiança em cada ponto da curva, todas as execuções devem ter o mesmo número de avaliações naquele ponto. Isso é trivial para algoritmos com orçamento fixo (busca aleatória, SA com critério de parada por número de avaliações), mas requer atenção em algoritmos adaptativos que podem terminar em iterações distintas. Neste curso, fixaremos sempre um **orçamento de avaliações** uniforme para garantir que as curvas sejam comparáveis entre execuções.

3.3 Testes estatísticos e o Problema das Múltiplas Comparações

Comparar apenas médias de $\overline{f^*}$ é insuficiente e pode induzir conclusões equivocadas. A análise estatística correta depende do número de algoritmos comparados.

3.3.1 Comparação entre dois algoritmos

Teste de Wilcoxon com postos sinalizados (pareado): compara dois algoritmos A e B sobre o mesmo conjunto de instâncias. É o teste padrão para comparação pareada na área e não assume normalidade dos dados.

Procedimento recomendado:

1. Formule a hipótese nula H_0: "os algoritmos A e B têm desempenho estatisticamente equivalente".
2. Execute $N_r \geq 30$ rodadas independentes de cada algoritmo, com sementes aleatórias distintas e registradas.
3. Aplique o teste de Wilcoxon com nível de significância $\alpha = 0.05$.
4. Se $p < 0.05$, rejeite H_0. Reporte o valor-p **e** o tamanho do efeito ($r = \frac{Z}{\sqrt{N_r}}$), não apenas a decisão binária. Tamanhos de efeito pequenos ($r < 0.1$) podem ser estatisticamente significativos mas operacionalmente irrelevantes.

3.3.2 Comparação entre múltiplos algoritmos

Ao comparar $k > 2$ algoritmos, **não** aplique Wilcoxon repetidamente para cada par. Realizar $\binom{k}{2}$ testes com $\alpha = 0.05$ cada inflaciona severamente o erro tipo I: para $k = 5$ algoritmos, a probabilidade de ao menos um falso positivo sobe para $1 - 0.95^{10} \approx 40\%$.

O procedimento correto:

1. **Teste de Friedman** (extensão para $k > 2$ algoritmos sobre múltiplas instâncias, baseado em ranqueamento): testa a hipótese nula de que todos os algoritmos têm desempenho equivalente.
2. Se Friedman rejeita H_0 ($p < 0.05$), aplique teste **post-hoc com correção de Bonferroni-Dunn ou Nemenyi** para identificar quais pares diferem significativamente. Esses procedimentos controlam o erro tipo I da família de comparações.
3. Reporte o diagrama de ranqueamento crítico (Critical Difference diagram), que visualiza quais grupos de algoritmos não diferem significativamente entre si.

> ⚠️ **Atenção**
> **Inflação do erro tipo I é o erro estatístico mais comum** em comparações de metaheurísticas na literatura. Sempre que comparar mais de dois algoritmos, use Friedman + post-hoc com correção para múltiplas comparações. Ver Derrac et al. (2011) para um tutorial completo sobre o protocolo correto.

> **Exemplo**
> **Comparação entre busca aleatória e heurística gulosa no Problema da Mochila** ($n = 20$ itens, $W = l\lfloor 0\{, \}5 \sum w_i r \rfloor$, instância do Exercício 2 da Aula 1).
>
> Após 30 execuções independentes da busca aleatória (10^4 avaliações cada) e uma execução da heurística gulosa:
>
> | Método | $\overline{f^*}$ (valor) | σ_{f^*} | Melhor | Taxa de sucesso |
> |---|---|---|---|---|
> | Busca aleatória | — | — | — | — |
> | Heurística gulosa | — | n/a | — | n/a |
>
> Os valores exatos dependem da instância gerada — este é o experimento do Exercício 2 da Aula 1, que você já implementou. Ao aplicar o teste de Wilcoxon entre as 30 execuções da busca aleatória e o valor único da heurística gulosa, o que o valor-p diz sobre a superioridade estatística de cada método? O tamanho do efeito é grande o suficiente para ser **operacionalmente** relevante, além de estatisticamente significativo?

> Este exemplo ilustra que "melhor algoritmo" é sempre uma afirmação condicional: depende da instância, do orçamento de avaliações e do critério de comparação — exatamente o que o NFL prevê.

3.4 Custo de Avaliação e Critério de Parada

O custo de avaliar $f(\boldsymbol{x})$ varia enormemente entre problemas e é uma das variáveis mais importantes no projeto de experimentos. Alguns exemplos ilustram a diversidade:

- **Problema da Mochila** ($n = 20$): avaliar f custa $O(n)$ — microssegundos. O número de avaliações de função 10^4 avaliações do Exercício 2 da Aula 1 é computacionalmente factível.
- **Empacotamento 3D com restrições físicas**: avaliar se um arranjo de caixas é estável sob empilhamento requer simulação de contato ou verificação geométrica recursiva — potencialmente centenas de milissegundos por avaliação. Um número de avaliações de função de 10^4 avaliações pode representar horas de processamento.
- **Simulações de engenharia** (fluxo de potência, dinâmica de fluidos, elementos finitos): uma única avaliação pode levar minutos ou horas, reduzindo o limite prático a dezenas ou centenas de avaliações totais.

Essa assimetria tem consequência direta na escolha do método: algoritmos populacionais (PSO, AG) realizam múltiplas avaliações **por iteração**, enquanto algoritmos de trajetória (SA, Busca Tabu) realizam uma. Comparar número de **iterações** penaliza injustamente os métodos populacionais — e comparar tempo de CPU confunde custo de avaliação com eficiência de implementação.

A prática recomendada (adotada nos benchmarks IEEE CEC e nas competições de otimização de caixa-preta) é usar o **número total de avaliações de f** como unidade de custo universal, fixando um **número de avaliações de função** (*evaluation budget*, ou NFE — *number of function evaluations*) idêntico para todos os métodos. Essa escolha é independente do hardware, da linguagem de implementação e do custo intrínseco de f.

Comparações em **tempo de CPU** são relevantes quando a aplicação tem restrições de latência reais. Nesse caso, detalhes de implementação (linguagem, paralelismo, vetorização) tornam-se parte da solução, não artefatos a controlar — e devem ser reportados explicitamente.

Protocolo recomendado: fixe o número de avaliações de função de avaliações como critério primário de parada; reporte também o tempo de CPU médio por execução com desvio padrão, especificando hardware e linguagem. Quando os dois critérios levarem a conclusões conflitantes, discuta explicitamente a razão.

4 Exercícios

Exercício

1 (Análise Experimental) - *1,0 ponto.* Implemente em Python o seguinte experimento:

(a) Defina $f(\boldsymbol{x}) = \sum_{i=1}^{10} x_i^2$ com $x_i \in [-5.12, 5.12]$ (Esfera — unimodal) e $g(\boldsymbol{x}) = \sum_{i=1}^{10} [x_i^2 - 10\cos(2\pi x_i) + 10]$ (Rastrigin — multimodal, 10 variáveis).

(b) Aplique busca aleatória pura (100 000 avaliações, 30 execuções com sementes registradas) em ambas as funções.

(c) Calcule $\overline{f^*}$, σ, melhor e pior resultado para cada função.

(d) Plote a curva de convergência **média** com intervalo de confiança de 95% em cada ponto do eixo de avaliações. Justifique por que o orçamento fixo é necessário para que o IC seja computável uniformemente ao longo da curva.

(e) O que os resultados sugerem sobre a dificuldade relativa das duas funções para busca aleatória? Como isso se relaciona com o NFL?

Exercício

2 (Estatística) - *1,0 ponto.* Em um estudo comparativo, o algoritmo A produz $\overline{f^*} = 1045$ (menor é melhor) e o algoritmo B produz $\overline{f^*} = 1052$, ambos com 30 execuções. Um colega afirma: "A é 0,67% melhor que B, portanto A deve ser adotado". Critique essa afirmação identificando ao menos três problemas metodológicos. Que informações adicionais você precisaria para uma conclusão válida? Esboce o protocolo experimental correto, incluindo o teste estatístico apropriado, o tratamento do problema de múltiplas comparações (caso outros algoritmos estejam sendo avaliados simultaneamente), e as métricas a reportar.

Leituras recomendadas:

• Wolpert, D. H., Macready, W. G. No free lunch theorems for optimization. **IEEE Transactions on Evolutionary Computation**, 1(1), 67–82, 1997.
• Blum, C., Roli, A. Metaheuristics in combinatorial optimization: Overview and conceptual comparison. **ACM Computing Surveys**, 35(3), 268–308, 2003. — Referência fundamental para taxonomia e análise comparativa de metaheurísticas.
• Talbi, E.-G. **Metaheuristics: From Design to Implementation**. Wiley, 2009. Caps. 1–3.
• Derrac, J., García, S., Molina, D., Herrera, F. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. **Swarm and Evolutionary Computation**, 1(1), 3–18, 2011. — Leitura obrigatória para o protocolo experimental desta disciplina.